

오픈 언어모델 Fine-Tuning 이론

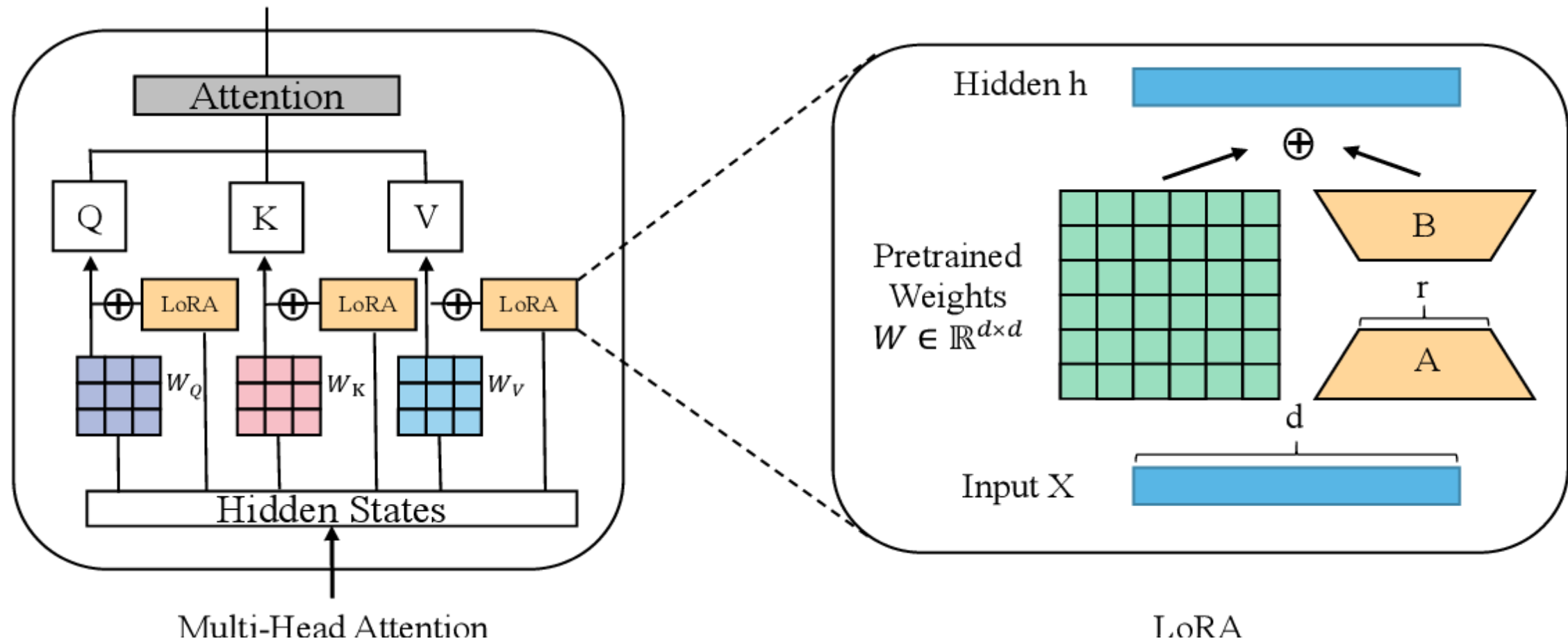
1. Fine-Tuning 의 개념

사전 학습된 모델을 특정 도메인이나 서비스 상황에 맞게 최적화하는 방법

새로운 데이터셋에 대한 특성을 사전 학습된 모델을 통해 추출하고, 모델의 출력 층을 새로운 작업에 맞게 조정한 후, 전체 모델을 새로운 데이터셋에 맞게 재학습하는 과정

2. Fine-Tuning 절차

사전 학습된 모델 로드 → 새로운 데이터셋의 특성 추출 → 새로운 분류기 추가 → 미세조정



3. Fine-Tuning 이 필요한 상황

1. 커스터마이즈

특정 도메인이나 서비스 상황에 최적화 해야 할때

2. 부족한 라벨 데이터 보완

사전 학습된 모델의 유용한 하위계층 데이터를 활용

4. Fune-Tuning의 어려운점

1. 도메인 차이(Domain Shift)

- 사전 학습된 모델이 학습한 데이터와 새로운 데이터 간에 차이가 클 경우, 모델이 기대한 대로 동작하지 않을 수 있음.
- 예를 들어, 자연 이미지로 학습된 모델을 의료 이미지에 적용하려 할 때, 데이터 특성이 크게 다르면 성능이 저하됨.

2. 과적합(Overfitting)

- 새로운 데이터셋이 작을 경우, 미세 조정 과정에서 모델이 과적합될 위험이 높아짐
- 모델이 새로운 데이터셋에 너무 맞추어져, 일반화 능력이 떨어짐.

3. 학습률 조절

- 학습률을 잘못 설정하면, 기존의 학습된 가중치가 크게 변하거나, 학습이 제대로 되지 않을 수 있음.
- 너무 높은 학습률은 모델을 불안정하게 만들고, 너무 낮은 학습률은 학습 속도를 느리게 만듦

4. 컴퓨팅 자원과 시간

- 큰 모델을 미세 조정하는 데는 많은 계산 자원과 시간이 필요함.
- 특히 대규모 모델의 경우, 하드웨어 요구사항이 매우 높아 개인/중소 규모에서 수행하기 어려움.

5. PEFT(Parameter Efiicient Fine Tuning) 의 장점

1. 효율적인 학습

- PEFT는 모델의 일부 파라미터만 조정하기 때문에, 전체 모델을 학습하는 것보다 훨씬 효율적
- 메모리 사용과 계산 자원을 절약할 수 있어, 더 적은 자원으로도 높은 성능을 달성 가능

2. 저비용 고효율

- 적은 학습 데이터와 자원으로도 모델을 효과적으로 미세 조정 가능
- 대규모 언어 모델과 같이 큰 모델을 사용할 때 유리

3. 빠른 학습

- 특정 파라미터만 조정하므로, 학습 속도가 빠름.
- 실시간 응용에도 적합

4. 적응력

- 모델의 특정 부분을 조정하는 다양한 방법을 제공되고 있어, 다양한 도메인과 작업에 Adapting 할수 있음.
- Adapters, Low-Rank Adaptation (LoRA), Prefix-Tuning 등의 기법을 통해, 필요한 부분만 조정 가능.

5. 일관된 성능과 안정성

- PEFT는 모델의 주요 특성을 유지하면서도 특정 작업에 맞는 조정을 할 수 있어, 안정적인 성능을 제공
- 과적합의 위험을 줄이고, 일반화 능력을 유지하게 됨.

6. LLM 분야의 대표적 PEFT 방법론

- 참고 문헌
 - <https://medium.com/@abonia/llm-series-parameter-efficient-fine-tuning-e9839fae44ac>
 - <https://lightning.ai/pages/community/article/lora-llm/>
 - <https://huggingface.co/docs/transformers/peft>
 - <https://github.com/Lightning-AI/lit-llama/> (fine-tune 구현 코드)

(1) LoRA(Low Rank Adaptation)

- 논문

LoRA(Low-Rank Adaptation of Large Language Models) [논문링크](#)

- LoRA 성능

LoRA can even outperform full finetuning training only 2% of the parameters

	Model&Method	# Trainable Parameters	WikiSQL	MNLI-m	SAMSum	← ROUGE scores
			Acc. (%)	Acc. (%)	R1/R2/RL	
Full finetuning →	GPT-3 (FT)	175,255.8M	73.8	89.5	52.0/28.0/44.5	
Only tune bias vectors →	GPT-3 (BitFit)	14.2M	71.3	91.0	51.3/27.4/43.5	
Prompt tuning →	GPT-3 (PreEmbed)	3.2M	63.1	88.6	48.3/24.2/40.5	
Prefix tuning →	GPT-3 (PreLayer)	20.2M	70.1	89.5	50.8/27.3/43.5	
	GPT-3 (Adapter ^H)	7.1M	71.9	89.8	53.0/28.9/44.8	
	GPT-3 (Adapter ^H)	40.1M	73.2	91.5	53.2/29.0/45.1	
	GPT-3 (LoRA)	4.7M	73.4	91.7	53.8/29.8/45.9	
	GPT-3 (LoRA)	37.7M	74.0	91.6	53.4/29.2/45.1	

Table 4: Performance of different adaptation methods on GPT-3 175B. We report the logical form validation accuracy on WikiSQL, validation accuracy on MultiNLI-matched, and Rouge-1/2/L on SAMSum. LoRA performs better than prior approaches, including full fine-tuning. The results on WikiSQL have a fluctuation around $\pm 0.5\%$, MNLI-m around $\pm 0.1\%$, and SAMSum around $\pm 0.2/\pm 0.2/\pm 0.1$ for the three metrics.

- **LoRA 란?**

- LoRA는 Low-Rank Factorization 방법을 활용해 LLM의 Linear Layer에 대한 업데이트를 근사화하는 기술.
- 기본 원리는 모델의 고차원 파라미터를 저차원 공간으로 투영한뒤, 저차원 공간에서의 추가 파라미터만 학습하고 이를 원래 모델에 병합하여 출력을 조절.
- 고차원 파라미터 대신 저차원 파라미터를 학습하므로, 필요한 메모리와 계산 자원이 크게 줄어들어, 모델의 최종 성능에 거의 영향을 주지 않으면서 훈련 속도를 높임

- LoRA 알고리즘 sudo 코드

(Transformer 예시)

- 1 원래 가중치 행렬 W : 크기가 $d \cdot k$ 인 행렬
- 2 저차원 행렬 A : 크기가 $d \cdot r$ 인 행렬로, 저차원 공간으로 투영.
- 3 저차원 행렬 B : 크기가 $r \cdot k$ 인 행렬로, 다시 고차원 공간으로 변환.
- 4 학습: 학습 과정에서 A 와 B 행렬을 학습. 이때 r 은 d 나 k 보다 훨씬 작기 때문에, 필요한 파라미터 수가 크게 줄어듬.
- 5 출력 계산: 학습된 저차원 행렬을 사용하여, 원래 가중치 행렬의 출력을 조정.

$$W + \alpha \cdot (A \cdot B)$$

(2) Prompt Tuning

- 논문

[The Power of Scale for Parameter-Efficient Prompt Tuning 논문링크](#)

- 모델의 가중치를 변경하지 않고, 입력 데이터에 대한 임베딩을 수정하여 모델이 원하는 작업을 수행하는 방법

- 엔지니어링 수행 절차

1 기존 모델 준비: 미리 학습된 대형 언어 모델(예: GPT, BERT)을 사용합니다.

2 저차원 행렬 **A**: 크기가 $d \cdot r$ 인 행렬로, 저차원 공간으로 투영.

3 소프트 프롬프트 생성 (자연어가 아닌 특정 목적에 최적화된 훈련가능한 임베딩 벡터)

- 모델의 입력 임베딩 차원과 동일한 차원의 임의의 초기값을 가진 소프트 프롬프트 벡터를 생성.

4 입력 데이터와 소프트 프롬프트 결합

- 입력 데이터 앞에 소프트 프롬프트 벡터를 추가하여 모델에 입력.

5 학습 과정

- 모델을 학습하면서 소프트 프롬프트 벡터를 최적화. 이때 모델의 나머지 부분은 고정된 상태로 유지.

- 일반적인 뉴럴 네트워크 최적화 방법(SGD)을 사용해 역전파 학습

(3) Prefix Tuning

- 논문

Prefix-Tuning: Optimizing Continuous Prompts for Generation

- 방법론

- 기존의 미세 조정 방법들과는 달리, Prefix-Tuning은 모델의 모든 가중치를 학습하는 대신 입력 데이터에 추가적인 벡터(프리픽스)를 붙여서 학습하는 방법
- 사전 학습된 언어 모델의 가중치는 고정(frozen)된 상태로 유지시킴. 대신, 프리픽스 벡터만을 추가로 학습시킴. 이를 통해 전체 모델의 가중치를 재학습하는 부담을 줄임
- 예를 들어, 입력 문장이 "This movie is great"라면, 프리픽스 벡터가 추가된 입력은 "[프리픽스 벡터] This movie is great"가 되는 것임.

방법론 비교

특징	Prompt Tuning	LoRA	Prefix Tuning
모델 파라미터	고정	저순위 행렬만 학습	고정
추가 요소	프롬프트 텍스트	저순위 행렬	Prefix 임베딩
학습 파라미터 수	적음	중간	적음
적용 방식	입력 변형	모델 가중치에 저순위 행렬 추가	입력 확장
유연성	높음	중간	중간

Prefix-Tuning 실습

오픈 언어모델 Fine-Tuning 실습

